

WEAREDEVELOPERS × BAND · TRACK POCKETFUL · TEAM REVENUEFLOW

Dark Factory

Three coding agents in one BAND room, building a payments service. Two of the three are forbidden from writing code.

147/147

shipped checks, isolated mode

3 h 49 m

one session, no restarts

11.7 M

tokens, measured in the log

\$11.70

of a \$25 credit

Grading is literal, so speed is the wrong advantage

Exact field names, exact status codes, exact error codes. A correct-looking service with one mistyped attribute scores nothing. Building it faster does not help.

The bottleneck is not implementation speed. It is refusing to hand off anything that does not conform.

What that buys

The crew ratio follows: one implementer, one verifier, one planner who also holds the gate. Every change passes a customs post before it counts.

What it costs

Throughput. On work that is already correct the checks add latency and buy nothing. We took the trade because the failure we feared — green tests, wrong names — is silent.

The band

Seat	Harness and model	Role	Writes code
coordinator	OpenCode · GLM-5.2	Plans, splits the work, routes ambiguities, runs the gate	No
implementer	OpenCode · GLM-5.2	The Go service, the Dockerfile, the tests	Yes
reviewer	OpenCode · GLM-5.2	Conformance against the specification text, never the report	No

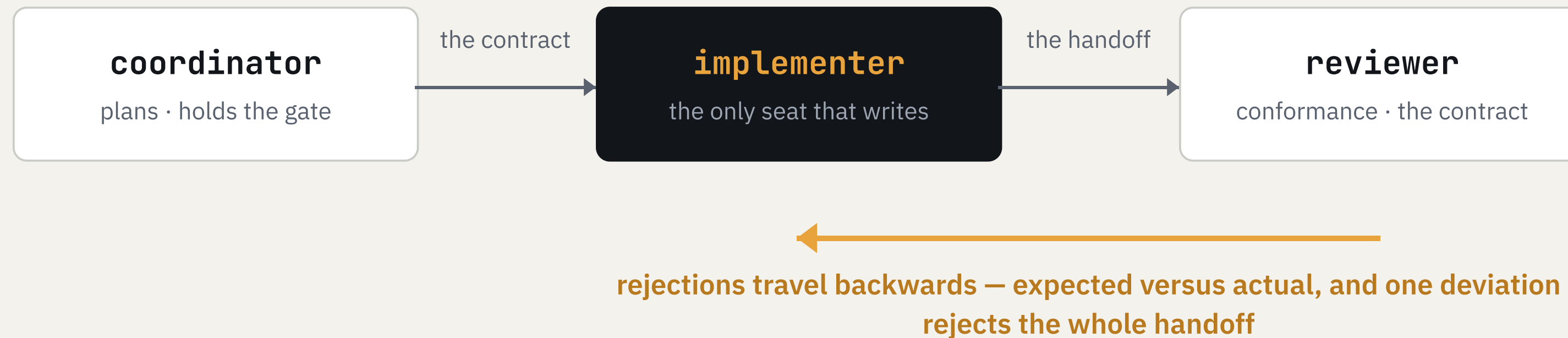
The mandate is the seat

The runner loads **mandates/<seat>.md** and passes it as that seat's system prompt. It is not documentation. Change the file and the agent behaves differently on its next turn.

Same model, different jobs

One model behind all three seats. The difference between them is entirely the mandate — which is the claim the rubric asks you to make reusable, and here it is true by construction.

Who talks to whom



Adding a seat to a room does not wake it — a message must mention it. So every handoff is an explicit **@mention**, and that is the wiring. Handoff targets are never hardcoded: each mandate tells the seat to inspect the room participants at handoff time and mention whoever holds the next role. The line can be re-crewed at runtime without editing a mandate.

One run, end to end

Time	Who	What happened
20:06	owner	Posts the task to the coordinator. This is the kickoff, and the only message that is not a correction.
20:12	coordinator	Publishes the plan and architecture, opens 17 board tasks, routes four spec ambiguities for adjudication.
20:29	reviewer	Publishes a 158-item conformance checklist, derived from the specification text only. Flags 22 items no shipped test would catch.
20:33	implementer	Accepts the checklist as its contract — before writing a single line of code.
22:41	implementer	Reports a clean build in the container and the C-items covered. Commit 5fa3caa.
22:46	owner — decision	Runs the gate independently. 146 of 147. Returns two defects with expected versus actual.
23:50	owner	147 of 147 against 662893a. Accepted.

One seat corrected another, before any code existed

coordinator · 20:12

Ambiguity A3 — an empty body on paying a request. Recommend: treat it as an empty object, default public.

reviewer · 20:29

Confirm, **with caveat**. The canonical stored body for idempotency comparison must be the empty object so an empty-body replay matches. A body of visibility public is a different JSON value — reusing the key across the two is 409 idempotency_key_reuse.

The coordinator answered what the endpoint should **accept**. The reviewer caught that it was silently wrong about what the endpoint should **store** — and that storing the normalised form would make a legitimate retry read as key reuse.

Both caveats became binding numbered items — **C-99**, **C-157** — in a 158-item contract the implementer acknowledged four minutes later.

Two of those items are structural — and checkable today

C-155 · no float64

Amounts arrive as 1000, 1000.0 and 1e3 — all valid, all equal. Decode through json.Number and math/big, never a float.

Result: zero matches across all 23 Go files.

C-152 · one chokepoint

A settlement that debits in a per-transfer loop passes the preview tests and loses conservation under concurrency.

Result: applyDelta declared once, three call sites, all in one file.

```
func settleBatch(users []*User, deltas []int64) *apiError {
    for i := range users {
        if users[i].Balance+deltas[i] < 0 {
            return errInsufficientFunds
        }
    }
    for i := range users {
        applyDelta(users[i], deltas[i])
    }
    return nil
}
```

Architecture amended to both points at commit 83f9dde, before implementation began.

What the factory did not catch

The reviewer was stopped at 20:34:40 to conserve budget, and never ran the verification pass it had committed to twelve minutes earlier.

Defect 1

Exported empty collections marshalled as null instead of an empty array, so a round trip through import failed.

Defect 2

The password hash was base64 on export and read as raw text on import. **Nobody could log in after an import.**

Who caught them

The owner, running the gate at 22:46. Not the reviewer. Fixed in 662893a with three regression tests.

The C-152 and C-155 properties hold. They hold because the implementer built to the contract — not because anyone verified it at the handoff.

Four inference paths failed before one worked

Path	Where it broke
Claude Code as the harness	Requires a paid plan or API balance. We had neither. This forced everything below.
OpenRouter free tier	The model was retired mid-setup — and the permission classifier runs on the same route, so the harness stops rather than degrades.
Groq behind LiteLLM	Every large-context model there is a reasoning model. The harness replays reasoning_content on the next turn and Groq rejects its own output.
Featherless behind LiteLLM	The Anthropic-to-OpenAI translation drops tool definitions . A coding agent with no tools reads and writes nothing.

What worked

OpenCode speaks OpenAI format natively and Featherless serves OpenAI format, so the translator leaves the path entirely. Every failure above is a failure of a translation layer, not of a model.

Measured, from the room log itself

Every seat runs with usage reporting on, so each turn's token count sits in the log next to the work that turn did. This is that log, summed — not the provider dashboard, which covers the whole account.

3 h 49 m

wall clock, one session

451

messages, 206 tool calls

11,743,469

tokens, 13 reported turns

\$11.70

of a \$25 sponsor credit

Seat	Turns	Input	Output
implementer	7	9,119,545	75,266
coordinator	3	1,938,353	122,724
reviewer	3	472,656	14,925

54 : 1

Input outweighed output fifty-four to one

A seat re-reads the accumulated room on every turn, so a factory costs what its own history costs — not what it produces. The implementer spent 9.1 million input tokens to emit 75 thousand, paying over and over to re-read a conversation it had mostly written itself.

What it built: pocketful

147/147

shipped checks, isolated mode

A wallet and payments service. Claimed stage 1, highest contiguous stage 1, run against commit b4af9b5. A single static Go binary in a scratch image, no outbound network at runtime.

Money must never be created, destroyed, or spent twice. Three invariants carry it.

Conservation

The sum of all balances is constant across any storm of concurrent operations.

Non-negativity

No balance goes below zero under any interleaving, not even transiently.

At most once

An idempotency replay returns the stored outcome and moves no additional money.

All amounts are exact integer counts of minor units. Rounding lives in exactly one function.

The delete test

Take BAND out and this factory does not degrade. It disappears.

The rejection has no channel

A bounce is a message that wakes one specific seat. Without mention routing there is nowhere for expected-versus-actual to go.

The contract has nowhere to live

The 158-item checklist was published to the room and read by the seat that did not write it. A pipeline has no shared surface for that.

The board is not a log

Seventeen tasks, claimed and closed by the seats themselves. Evidence a judge can read without trusting anyone's summary.

The human stops being a peer

Five steering messages went into the room as messages. Outside it, each one is a restart.

The limitation we declare

Permissions, the obvious one

The seats accept approvals automatically. That is what makes the run unattended, and it means a seat executes commands without asking anyone. The blast radius is the working directory — which is why the result repository is separate and credentials live outside it.

Judgment, the real one

A seat that reports work it did not do will be believed by the other seats unless one of them checks. We hit this twice: a file reported on disk three minutes before it existed, and a clean-build report that said nothing about two defects it had not thought to look for. Neither seat lied.

And the run was not unattended

Six human messages: one kickoff and five corrections. Three of the five are now standing sections in the mandates — derive requirements from the spec, never from test files; build in the container, never on the host; no seat idles through a stretch where it has a role. A correction the owner has to give twice is a missing mandate.

The verifier is the only part of this factory that was optional, and it is the part that was cut. The result passes 147 of 147 — and we cannot claim the factory is why.

Everything on this deck is in the repository

github.com/yanerox69/dark-factory-pocketful

FACTORY.md

The factory, measured.
Every number on this deck,
with the method that
produced it.

room.json

The whole room log,
downloaded unedited. 451
messages, including the six
that are ours.

mandates/

Three files, each loaded as a
seat's system prompt.
Nothing in them names this
track.

stage-1/

The service. Twenty-three
Go files, a Dockerfile, and
one command in RUN.md.

What we would change: fund the verifier first. It was the cheapest seat in the room at 472 thousand input tokens, and it was the one we switched off.